Why Did It Move?

Automated Root-Cause Analysis on ClickHouse

CH-Minds | Click-a-thon 2026 | InMobi track

"From alert to answer: the automated root-cause analyst."

Thousands of segments. One question: why did it move?

- InMobi's ad business spans thousands of app x device x geo x advertiser combinations.
- When revenue or fill rate moves, the answer today comes from a human manually drilling through dashboards, dimension by dimension.
- That doesn't scale, and it's slow even when it works - hours of investigation for a number that's already sitting in the data.
- The ask: detect the deviation, drill to the exact responsible segment, and explain it in seconds - with every number real and reproducible.

Detect -> Investigate -> Narrate. ClickHouse computes, the LLM only narrates.

- Detect - a background scan sweeps every headline metric x every dimension for deviations from a trailing same-weekday median baseline.
- Investigate - decomposes revenue into Requests x Fill rate x eCPM, ranks segments by deviation, then checks for a sharper two-dimension combination.
- Narrate + trace - the finished, computed result goes to the LLM once for plain-English phrasing. The LLM never queries ClickHouse and never sees a raw row.

Every number in the diagnosis traces back to a ClickHouse query result - zero hallucinated figures.

THE ACTUAL RESULT

Live on the unseen incident: revenue up 92%, hidden under a flat headline

- 2026-07-07 (unseen slice, released to all teams simultaneously): overall revenue +2.1% - unremarkable on the surface.
- Segment ranking finds country = CA at +92.5% (baseline $58.69 -> actual $112.99).
- Refined one level deeper: device_model = Pixel 8 within Canada, +117.7% - a sharper, more specific culprit.
- The same pattern held the day before too (2026-07-06: CA +91.5%, Pixel 8 +113.5%) - a genuine two-day-sustained anomaly, not noise.
- Requests, fill rate, render rate, and eCPM all checked and ruled out; every other dimension explicitly checked with nothing else standing out.

Trustworthy by construction: ruled-out list + full Langfuse trace

- Every factor and dimension checked - not just what was flagged - is recorded and shown as "checked and ruled out."
- Every /api/investigate and /api/scan call is wrapped in a Langfuse trace with real input/output at every span: day-coverage check, threshold computation, factor decomposition, segment ranking, combo refinement, narration.
- A judge can open the trace and audit exactly what the system checked, in what order, independent of the prose.
- Confidence score is computed (deviation magnitude vs threshold), never a hardcoded number.

Architecture: raw events -> rollup -> analysis -> narration -> trace

- ad_events (10.5M rows, ClickHouse Cloud) -> hourly_segment_metrics, an AggregatingMergeTree rollup populated by a materialized view at insert time.
- ORDER BY covers all 10 grouping dimensions, not a subset - for AggregatingMergeTree, ORDER BY is a row's merge identity.
- FastAPI backend (Railway) runs every detection/ranking/drill-down query against the rollup, never the raw fact table.
- React frontend (Vercel) - metric tree, flagged anomalies, hour-level drill-down, playback timeline, chat.
- Langfuse Cloud traces the full chain; the LLM (OpenAI, swappable) only narrates the finished JSON result.

CLICKHOUSE & OSS STACK

Where ClickHouse is doing the real work, not just storing data

- AggregatingMergeTree + State/Merge functions - fill rate, eCPM, CTR are sum/sum ratios that must never be averaged per-row; storing aggregate state lets any grain re-aggregate correctly later.
- Trailing-baseline window functions (ROWS BETWEEN N PRECEDING AND 1 PRECEDING, partitioned by segment + day-of-week) compute every segment's baseline in one query, not one round-trip per segment.
- A 36-45 query background scan (5 metrics x 9 dimensions) is embarrassingly parallel and reads only the rollup - never a full scan of the 10.5M-row fact table.
- Least-privilege access: a read-only ClickHouse user is the only credential any LLM-facing code path can reach.

Baseline design is the difference between a real alert and a false one

- Trailing same-weekday MEDIAN (quantileExact), not a flat average and not a mean.
- A flat average false-alarms every weekend - measured: weekday revenue averages $521 vs $397-423 on weekends, a genuine ~20% seasonal dip.
- A mean baseline lets a real incident poison its own future comparisons - measured directly: a genuine -44.8% incident inflated the following Sunday's reading from a true +5.5% to a false +22.7% under a mean baseline.
- Detection requires BOTH a large percentage deviation AND a real z-score (AND, not OR) - the OR version flagged ~26% of everything; the AND version flags ~1%, empirically justified against the dataset's own noise distribution.

Factor decomposition + segment ranking + combo refinement

- Revenue is decomposed via its real formula: Requests x Fill rate x eCPM - the diagnosis names WHICH factor moved, not just that revenue moved.
- Every one of 9 business dimensions is ranked by deviation to find the responsible segment.
- A hierarchical second pass checks whether a two-dimension combination (e.g. country + device_model) deviates even more sharply than the marginal segment alone.
- Same mechanism drills to hour grain - an incident concentrated in a few hours isn't diluted by averaging across a whole day.

Update-friendly: open sessions, new data, and re-scans are incremental

- New data lands via a staged load with an explicit overlap check - re-running the loader on already-loaded days refuses safely instead of silently double-counting (ad_events does not deduplicate).
- Dimension tables (apps/advertisers/geo_device) are ReplacingMergeTree, reloadable without a full rebuild.
- detect.scan() is idempotent by design - checks currently-open candidates before inserting, so re-running it does not re-flag what's already been found.
- Per-metric thresholds are computed live from whatever's currently loaded, not a constant calibrated once and forgotten - the system adapts as more data (like the unseen incident slice) arrives.

SCALABILITY & IMPACT

Scale framing: detection cost tracks dimension cardinality, not event volume

- Governing number: only ~767,984 dimension combinations actually occur in this dataset (vs 17.2M theoretical) - the space is unsaturated today.
- A day-grain rollup is capped at days x 767,984 rows regardless of event count - measured: 37x smaller than raw at 1B events, 372x at 10B, 3,717x at 100B.
- Partition pruning verified directly: a 35x wider date-range scan costs only 2.2x more, not 35x.
- The background scan is 45 fully independent queries - embarrassingly parallel, no shared state.
- LLM cost is O(1) in data volume - the model narrates one finished JSON object, it never sees a raw row.

Correctness was tested against real data, not assumed

- Found and fixed a genuine rollup corruption: an ORDER BY key missing 6 of 10 grouping columns let AggregatingMergeTree silently collapse dimensions on background merge - caught via an independent cross-check against raw ad_events, not by inspection.
- Found and fixed a partial-day comparison bug that would flip a genuine +19.6% into a fabricated -27.9% on a day released mid-day - exactly the shape the unseen incident could arrive in.
- Migrating to ClickHouse Cloud surfaced two more real bugs, both found and fixed: a non-atomic async-insert-into-materialized-view interaction that silently double-wrote data, and a scan() race that duplicated flagged candidates - both caught live, root-caused, and fixed in the deployed system.

THE UNSEEN INCIDENT

Built for the unseen incident - and it found real signal

- The real sealed unseen-incident dataset (2026-07-06 to 07-10, 1.5M events) is loaded and live on the hosted demo right now.
- Two independent, high-confidence findings, each run through the real pipeline with a captured Langfuse trace: revenue / country=CA -> device_model=Pixel 8 (sustained two days, confidence 1.0), and fill_rate / os_version=iOS 17.5 (confidence 1.0).
- Every diagnosis, number, and trace is reproducible live on the hosted demo - not a screenshot of a one-time run.

HOW WE BUILT IT

Stack

- ClickHouse Cloud - primary datastore and the engine doing every bit of the analysis.
- Langfuse Cloud - full pipeline tracing, meaningfully integrated (not a checkbox) - the direct mechanism for the traceability judging criterion.
- FastAPI (Python) backend on Railway; React + Vite + Tailwind + shadcn/ui frontend on Vercel.
- OpenAI (gpt-4o-mini) narration, behind a provider-agnostic interface that also supports Anthropic and Gemini.

Team & thank you

CH-Minds

Mohamed Hussain S

Ravivarman R

Praveen Kumar S

ClickHouse does the analysis. The LLM only narrates.

Every number reproducible, every run traced.